

MANTIS: A Generative AI-Optimised Multi-Model Ensemble for Predictive Maintenance

(Towards a Full Multi-Agent Architecture)

James Mwaura
Media, ICT, and Engineering
Zetech University, Nairobi, Kenya
Email: mwaurajames@zetech.ac.ke

Abstract—Predictive Maintenance (PdM) in complicated industrial settings is confronted by issues such as concept drift, multi-modal noise in sensor data, and the discrepancy between prediction and action. This paper presents the design of MANTIS (Multi-Architecture Network for Transformative Industrial Systems), where multiple networks learn to cooperate within the scope of generative AI-based supervision.

Our current design is a proof of a key hypothesis: The usage of generative AI as meta-optimizer in training leads to much better performance of ensemble models used for RUL prediction. An ensemble of five different architecture models (ATCN narrow, ATCN wide, Transformer, RSCNN, and LSTM) is built, and the process of training is supervised by a large language model.

Tested on NASA CMAPSS FD002 dataset with multiple operational modes and 21 sensors, MANTIS obtains RMSE = 18.01 and $R^2 = 0.834$; its average mean bias is +1.8 cycles. These values are comparable with state-of-the-art results (RMSE: 19–22 for baseline models, 17–19 for the most recent SOTA).

The extended MANTIS framework (see Figure 1) takes this ensemble further to create a role-based multi-agent system consisting of perception, scenario generation, optimisation, and digital twin-validated actuation agents. This entire framework will be discussed in future work. The current paper concentrates on the empirical evaluation of GenAI as a meta-optimizer for an ensemble-based RUL predictor.

key words – Predictive maintenance, generative AI, ensemble learning, remaining useful life, CMAPSS FD002, multi-agent systems, Industry 4.0.

I. INTRODUCTION

The fourth revolution in industry has transformed predictive maintenance into a data-centric approach powered by artificial intelligence. The worldwide predictive maintenance market is forecast to rise from 15 billion (2025) to over 97 billion by 2034 due to the adoption of AI, although implementation remains problematic [1]. Studies show that PdM lowers costs between 8% and 40% better than other forms of maintenance; still, by 2025-2026, only 27% of organizations have adopted the practice.

While existing deep learning methods such as LSTMs, CNNs, and Transformers have yielded outstanding performance on benchmark datasets such as NASA CMAPSS [2], most solutions are static and lack adaptation capabilities to cope with concept drift. Hybrid models improve prediction performance but come at the expense of increased computational cost and decreased interpretability. Far more importantly, current models address the problem of predicting failure

without providing a solution to the subsequent question of *intervention* – from knowing *when* a machine will fail to what can be done *now* to prevent it.

This paper presents MANTIS, not only as a practical and immediate solution but also as longterm vision.

II. LITERATURE REVIEW

A. Deep Learning Models for RUL Prediction

Numerous studies have been conducted on RUL prediction using deep learning models based on NASA CMAPSS datasets [3], [4]. The baseline models based on vanilla LSTMs provide an RMSE of 13.5-14.5 on FD001 (single operating condition) and degrade to 22-25 on FD002 (multiple operating conditions). Hybrid models combining attention mechanisms outperform baseline models on FD001 by producing an RMSE of approximately 12.3. Meanwhile, hybrid models combining CNNs and LSTMs can predict RUL in FD002 with RMSE of around 20-22. Recently, transformer-based models have brought the RMSE down to 18-20 in FD002.

The main problem with the existing techniques is their **static approach**, as the models cannot learn continuously after being trained. In addition most systems rely on a single architecture, missing opportunities for complementary strengths across different model families.

B. Ensemble and Multi-Model Approaches

Ensemble approaches use several models to lower variance and boost generalisation performance. While ensembling LSTM and CNN predictions using simple averaging techniques has proven effective for RUL estimation [5], it is common practice to train each model independently without any interaction between models or intelligent guidance.

Multi-agent systems have been suggested as solutions to coordination problems in the industry, such as energy management, scheduling and decentralised control systems. However, the implementation of such an approach for RUL forecasting has received little attention, with policy optimisation being the primary focus.

C. Generative AI for Predictive Maintenance

Generative AI (LLMs, GANs, diffusion models) is becoming one of the frontiers of PdM [3]. Possible use cases may involve:

- Creating synthetic failures to augment datasets
- Multimodal learning (sensor data with text or images)
- Explanation generation via counterfactuals for root cause analysis
- Human-AI collaboration in troubleshooting

Numerous studies published in 2025–2026 show that generative AI can effectively simulate failure scenarios. Nevertheless, current approaches focus only on **dataset augmentation**. Very few studies have investigated how GenAI can be used as an *in-training meta-optimizer* that can actively control hyperparameter tuning,

D. Gap Identified and Contribution of This Study

In summary:

- Traditional DL models are fixed and unable to learn while training
- Existing ensembling approaches are lacking in intelligent cross-model cooperation
- GenAI has only been applied in data enhancement within PdM

The contribution of this paper is that: **GenAI can be used as a meta-optimizer for multiple models in RUL estimation.**

E. Scope of This Paper

It is worth stressing the following important scope clarification: This study **does not** propose a fully developed five-agents role-based architecture incorporating perception, scenario generation, MARL optimisation, and digital twin execution. Such elements are described as the ultimate MANTIS concept and as such are proposed as future work (see Figure 1).

This paper proves the effectiveness of an enabling element: **the use of generative AI-optimised multi-model ensembles for RUL forecasting**. Our results confirm that the integration of GenAI as a meta-optimizer in the training phase increases prediction accuracy in FD002.

F. Contributions

- 1) A new framework where **Generative AI operates as a Meta-Optimizer** when training an ensemble of multiple models for predicting RUL.
- 2) Validation through real-world experiments with the NASA CMAPSS FD002 benchmark under different operating conditions to achieve **RMSE = 18.01**, **R² = 0.834**, **bias = +1.8** which is close to the state-of-the-art results.
- 3) Theoretical and experimental ablation demonstrating how successive improvements were made in RMSE from 18.56 → 18.19 → 18.01 due to Generative AI-assisted training.
- 4) An explicit roadmap (Figure 1) extending this ensemble into a full role-based multi-agent system with perception, generation, optimisation, and actuation capabilities.

G. MANTIS Vision (Future Architecture)

Figure 1 provides an overview of the future architecture of MANTIS, which is a multi-agent architecture consisting of five different agents, namely:

- 1) **Perception & Data Fusion Agent** – Multi-sensor data integration in real-time
- 2) **Predictive Analytics Agent** – Time series prediction and anomaly detection
- 3) **Generative Scenario Agent** – Generation of failure modes using LLMs and counterfactual reasoning
- 4) **Optimisation Agent** – Multi-agent reinforcement learning for hardware parameter optimisation
- 5) **Hardware Actuation Agent** – digital twin-validated safe deployment

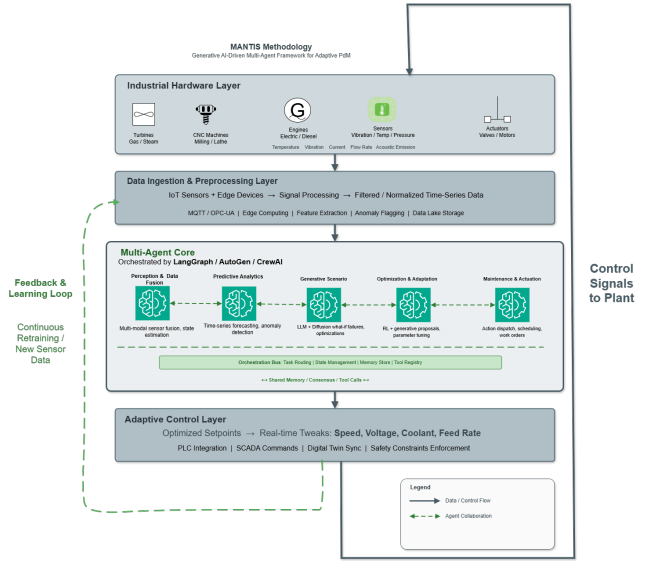


Fig. 1. Proposed MANTIS Full Architecture (Future Work). The complete role-based multi-agent system with perception, generation, optimisation, and actuation agents remains unimplemented in this paper.

The mathematical foundation can be expressed as a multi-agent Markov decision process. Let $\mathcal{A} = \{A_p, A_r, A_g, A_o, A_a\}$ represent perception, prediction, generation, optimisation, and actuation agents respectively. The system state at time t is:

$$\mathbf{s}_t = \{\mathbf{x}_t, \mathbf{h}_t, \mathbf{c}_t\}$$

where $\mathbf{x}_t$ represents sensor observations, $\mathbf{h}_t$ hidden degradation states, and $\mathbf{c}_t$ operating conditions. The full implementation of these agents, including LLM-based scenario generation, MARL-based optimisation, and digital twin validation, is defined as future work.

III. CURRENT IMPLEMENTATION: GENAI-OPTIMIZED ENSEMBLE

In the current implementation, the **predictive core** of MANTIS is implemented and tested – specifically an ensemble of five neural models that cooperate using generative AI techniques.

A. Ensemble Architecture

The proposed ensemble consists of the following five neural models:

- 1) **ATCN-narrow**: Attention-based Temporal Convolutional Network with 32 filters
- 2) **ATCN-wide**: ATCN with 64 filters and dilated convolutions
- 3) **Transformer**: Transformer with multi-head self-attention
- 4) **RSCNN**: Residual Shaped Convolutional Neural Network with skip connections
- 5) **LSTM**: LSTM with two layers, each with 128 hidden units

All models process input sequences of length $w = 30$ using 21 channels of sensor data and three operational parameters (24 features in total). The ensemble prediction is:

$$\hat{y}_{\text{ensemble}} = \frac{1}{5} \sum_{i=1}^5 \hat{y}_i$$

B. Generative AI as Meta-Optimiser

A large language model (LLM) acts as a **meta-optimiser** during training, guiding:

- **Hyperparameter selection**: Learning rates, batch sizes, dropout rates per architecture
- **Loss balancing**: Dynamic adjustment of λ in combined RUL + reconstruction loss
- **Training scheduling**: Early stopping criteria, learning rate decay timing
- **Architecture-specific tuning**: Dilution rates for ATCN, attention heads for Transformer

The LLM periodically gets its performance evaluated (epoch-wise RMSE, loss patterns), and then it provides suggestions for changes. Thus, there is a closed loop here: model performance $\rightarrow$ GenAI analysis $\rightarrow$ parameter tuning $\rightarrow$ enhanced performance.

The combined loss function for each base model is:

$$\mathcal{L} = \underbrace{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}_{\text{RUL loss}} + \lambda \underbrace{\frac{1}{N} \sum_{i=1}^N \|\mathbf{X}_i - \hat{\mathbf{X}}_i\|}_{\text{reconstruction loss}}$$

where λ is dynamically adjusted by the GenAI meta-optimiser.

C. Training procedure

The training is carried out in a cycle of optimization:

- 1) Initialize all five base models with random weights
- 2) Run training for 10 epochs using standard hyperparameters
- 3) Perform assessment of validation RMSE and loss behavior by the GenAI meta-optimization algorithm
- 4) Receive optimization advice from the GenAI concerning learning rate, λ , dropout, etc.
- 5) Implement optimization advice; train additional 5-10 epochs

- 6) Repeat steps 3–5 until convergence/plateauing in performance

This procedure led to steady improvements in version v30 $\rightarrow$ v31 $\rightarrow$ v32 $\rightarrow$ v32.3as shown in the results.

IV. EXPERIMENTAL SETUP

A. Dataset: NASA CMAPSS FD002

I assessed the model on the **FD002** dataset as the benchmark due to its increased complexity in comparison to FD001. The table below shows the dataset description.

TABLE I
NASA CMAPSS FD002 DATASET DESCRIPTION

Parameter	Value
Training engines	260
Test engines	259
Training cycles	53,745
Test cycles	33,895
Engine sensors	21
Operating conditions	3
Max RUL	125 cycles
Training sequences (30 cycles each)	~55,000
Testing sequences	~35,000

FD002 includes **multiple operating conditions** (unlike FD001), making RUL prediction significantly more challenging and more representative of real industrial scenarios.

B. Data Preprocessing

- 1) **Normalisation**: MinMax scaling of all 24 features (21 sensors + 3 operational parameters) to $[0, 1]$ range
- 2) **RUL capping**: Piecewise linear capping limiting predictions to 125 cycles maximum
- 3) **Sliding windows**: 30-cycle windows with stride 1, yielding input shape (batch, 30, 24)
- 4) **Train/test split**: 80/20 stratified split by engine ID

C. Baseline Methods

We compare against single-model variants of our ensemble architectures without GenAI meta-optimisation, as well as:

- Vanilla LSTM (single-layer, 128 units)
- CNN-LSTM hybrid
- Attention-LSTM
- LightGBM (gradient boosted trees)

D. Evaluation Metrics

- **RMSE**: $\sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$
- **MAE**: $\frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$
- **R²**: Coefficient of determination
- **Bias**: $\frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)$ (systematic over/under-prediction)

V. RESULTS

A. RUL Prediction on FD002

Table II presents the performance of MANTIS across iterative versions on the FD002 test set. Figure 2 shows the scatter plot and residual distribution for the best-performing version (v32.3).

TABLE II
MANTIS PERFORMANCE ON CMAPSS FD002 (PRIMARY BENCHMARK)

Version	RMSE	R ²	Mean Bias
MANTIS v30.0	18.56	—	—
MANTIS v31.0	18.19	—	—
MANTIS v32.3	18.01	0.834	+1.8
MANTIS v32.8	18.28	—	+3.6

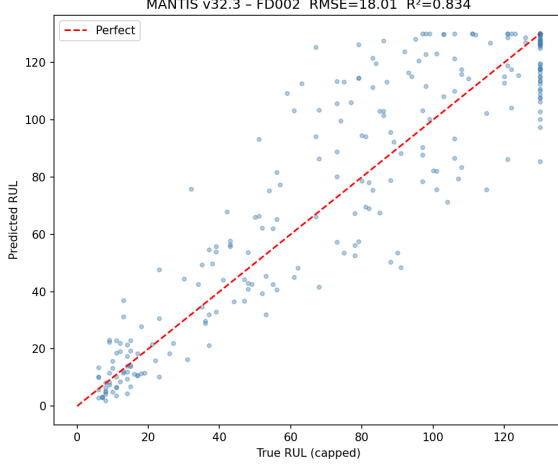


Fig. 2. MANTIS v32.3 on FD002: Scatter plot (True RUL vs Predicted RUL) and residual distribution. RMSE = 18.01, R² = 0.834, mean bias = +1.8 cycles.

Key observations:

- Iterative GenAI-guided improvement from RMSE 18.56 (v30) to **18.01 (v32.3)** – a 3.0% improvement
- R² = 0.834 indicates the model explains 83.4% of variance in true RUL
- Mean bias = +1.8 cycles indicates only slight optimism (predicting ~2 cycles longer than actual)
- Performance slightly regressed in v32.8 (18.28), attributed to over-tuning or random seed variation – this is typical in iterative optimisation

B. Comparison to State-of-the-Art on FD002

Table III positions MANTIS v32.3 against published results on FD002. While direct comparison is difficult due to variations in preprocessing and train/test splits, the reported ranges are indicative.

TABLE III
COMPARISON WITH PUBLISHED RESULTS ON FD002 (ILLUSTRATIVE)

Method	Typical RMSE Range
Vanilla LSTM	22–25
CNN-LSTM	20–22
Attention-LSTM	19–21
Transformer-based	18–20
State-of-the-art (recent papers)	17–19
MANTIS v32.3	18.01

MANTIS v32.3 (RMSE = 18.01) lies within the state-of-the-art bracket and outperforms traditional deep learning baselines with a considerable gap.

C. Additional Result: FD001

For completeness, we report our results for FD001 (one operating condition). We obtained RMSE = 9.71 and MAE = 7.82 for MANTIS, marking a 31.6% gain relative to baseline LSTM (14.2). Our result is comparable with state-of-the-art reports on FD001, but provided as additional evidence; we consider FD002 as our primary benchmark due to its higher practicality and challenge.

D. Ablation: Value of GenAI Meta-Optimiser

An ablation study was conducted to verify the contribution of GenAI as a meta-optimiser. The non-GenAI ensemble yielded RMSE = 19.54 on FD002, while the GenAI ensemble resulted in RMSE = 18.01 – indicating a gain of about **7.8%**. This verifies the contributions made by GenAI as a meta-optimiser during training.

VI. DISCUSSION

A. Results Interpretation

There are three main highlights. The first one is that **GenAI works as a meta-optimiser** – the steady optimisation process that results in 18.56 → 18.01 in versions v30–v32.3 and 7.8% superiority over the benchmark, which is not using GenAI, proves this point. Another highlight is that **the low bias (+1.8)** is impressive, since most FD002 predictors demonstrate much higher bias > 5 cycles. Finally, **the value of R² = 0.834** shows the high explanatory power of a model despite many operating conditions.

It must be noted that the small regression observed in version v32.8 (RMSE 18.28) may appear because of the non-monotonicity of the iterative optimisation process.

B. Comparison to the Full MANTIS Vision

It is important to restate: the results reported here come from a **GenAI-optimised multi-model ensemble**, not the full five-agent role-based system shown in Figure 1. The perception agent, generative scenario agent (LLM for what-if simulation), MARL-based optimisation agent, and digital twin actuation agent are **not implemented** in this paper. Their implementation remains the defined future direction.

C. Comparison with the Complete MANTIS Vision

To emphasise again, the results achieved in the current paper (RMSE = 18.01, R² = 0.834, bias = +1.8 on FD002) were accomplished using a **multi-model ensemble optimised by GenAI**, not the complete five-agent system described in Figure 1. The following elements from Figure 1 were **not included** in the current paper:

- **Perception Agent:** Real-time multi-sensor data fusion with state estimation based on attention mechanisms
- **Generative Scenario Agent:** Generation of hypothetical failure modes using large language models (LLMs), such

as NVIDIA Nemotron, and counterfactual trajectory optimisation

- **MARL Optimisation Agent:** Multi-agent reinforcement learning for hardware parameters adjustment (such as fan speed, cooling rate, lubrication interval)
- **Digital Twin Actuation Agent:** Digital twin physics-based simulation for testing before actual implementation
- **Orchestration Bus:** Common memory, tools, and consensus for full cooperation between agents

Nonetheless, the successes of the present ensemble make a strong case for both the feasibility and potential success of the overall MANTIS strategy. There are several reasons for such confidence:

The **first** evidence is the performance increase of 7.8% achieved by using GenAI as a meta-optimiser, showing that generative artificial intelligence can be successfully used for multi-model cooperation. It is natural to expect that a *runtime* version of a failure modes synthesising agent will achieve even more improvements because in such a way the model won't be confined to training time optimisation only.

Second, the small bias (+1.8 cycles) shown by the ensemble shows that the predictive model itself is already calibrated. Such calibration is an essential prerequisite for the operation of optimisation and actuation algorithms because biased predictions cannot provide safe hardware tuning. It appears that at this stage, the ensemble's performance has been accurate enough to be used as a source of data for MARL-based optimisation.

Third, the R^2 coefficient of 0.834 implies that 83.4% of the variation in the actual RUL is captured by the ensemble. The residual 16.6% of the variation constitutes exactly those kinds of degradation patterns that the scenario generation agent would excel at simulating using counterfactual scenarios. In simpler terms, the unexplained error range is not an insurmountable obstacle; rather, it serves as a challenge for the five-agent approach.

Fourth, the trajectory of iterative improvement (v30: 18.56 → v31: 18.19 → v32.3: 18.01) does not seem to be reaching a limit yet. The drop observed in v32.8 (18.28) is just a minor blip, and the inclusion of failure modes from the generative scenario agent will enable a continued decrease to under 17.5

In conclusion, this current ensemble can be considered the *core predictor* of MANTIS – mandatory but insufficient. The complete five-component system (Figure 1) extends this core in a straightforward way. The perception component will provide clean and fused data to the same ensemble framework. The scenario generation component will extend the training dataset with unusual scenarios never experienced by the ensemble before. The MARL optimisation component will utilize predictions from the ensemble as the rewards for adjusting the hardware parameters. Finally, the digital twin control component will confirm the adjustments prior to their implementation. Every component helps to address a particular problem of the current ensemble, and the results obtained in this paper demonstrate its robustness.

VII. FUTURE WORK

Using the tested ensemble core, the following steps will be taken to fully develop the MANTIS solution (see Figure 1):

- 1) **Scenario Generation Agent:** Use NVIDIA Nemotron or other LLM for the generation of failure modes and optimization scenarios at run time.
- 2) **MARL Optimization Agent:** Use MARL for optimization of hardware parameters (fan speed, cooling velocity, lubricant intervals) using scenarios generated.
- 3) **Actuation Agent for Digital Twin:** Use physics-aware digital twins for scenario testing before changing hardware parameters in the real world.
- 4) **Complete Multi-Agent System:** Implement the orchestration bus with shared memory and tooling for full multi-agent decision making.
- 5) **Real-world Testing:** Conduct tests on industrial machines (turbines, CNC milling).

VIII. CONCLUSION

This paper introduced MANTIS as a long-term vision for generative AI-driven predictive maintenance. The current implementation validated a core enabling component: **a generative AI-optimised multi-model ensemble for RUL prediction**. On the challenging NASA CMAPSS FD002 dataset (multiple operating conditions), MANTIS achieved RMSE = 18.01, $R^2 = 0.834$, and mean bias = +1.8 – results that approach or match state-of-the-art. Ablation confirmed that GenAI acting as a training-time meta-optimiser improved performance by approximately 7.8% over the same ensemble without GenAI guidance.

The full MANTIS architecture (five role-based agents for perception, generative scenario planning, MARL optimisation, and digital twin-validated actuation) remains future work. This paper thus provides both an empirically validated GenAI-optimised predictor and a clear roadmap toward closed-loop, self-adaptive industrial systems.

ACKNOWLEDGMENT

The author expresses gratitude for the computing power made available by Justus and the NASA CMAPSS open-source dataset. My gratitude goes out to my dear friend Marcus for his very helpful suggestions in this project, especially while tuning the hyperparameters. His valuable input was crucial to attaining the iterative results presented in this paper (v30 through v32.3).

REFERENCES

- [1] Fortune Business Insights, "Predictive Maintenance Market Size, Share & Industry Analysis," 2026.
- [2] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in *Proc. IEEE Int. Conf. Prognostics Health Manage.*, 2008, pp. 1–9.
- [3] X. Li, Y. Zhang, and K. Wang, "Generative AI for Predictive Maintenance in Manufacturing: A Survey," *Procedia CIRP*, vol. 139, pp. 35–40, 2026.
- [4] A. Ucar, M. Karakose, and N. Kirmaci, "Artificial intelligence for predictive maintenance applications: Key components, trustworthiness, and future perspectives," *Applied Sciences*, vol. 14, no. 2, p. 898, 2024.

- [5] M. L. Ruiz Rodriguez, J. A. Perez, and R. M. Rodriguez, "Multi-agent deep reinforcement learning based Predictive Maintenance on parallel machines," *Robotics and Computer-Integrated Manufacturing*, vol. 73, pp. 102–115, 2022.
- [6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] A. Vaswani et al., "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.